arXiv:2410.11989v2 [cs.RO] 17 Oct 2024

Dynamic Open-Vocabulary 3D Scene Graphs for Long-term Language-Guided Mobile Manipulation

Zhijie Yan[1], Shufei Li[2], Zuoxu Wang[1]✉, Lixiu Wu[3], Han Wang[4], Jun Zhu[4], Lijiang Chen[4], Jihong Liu[1]

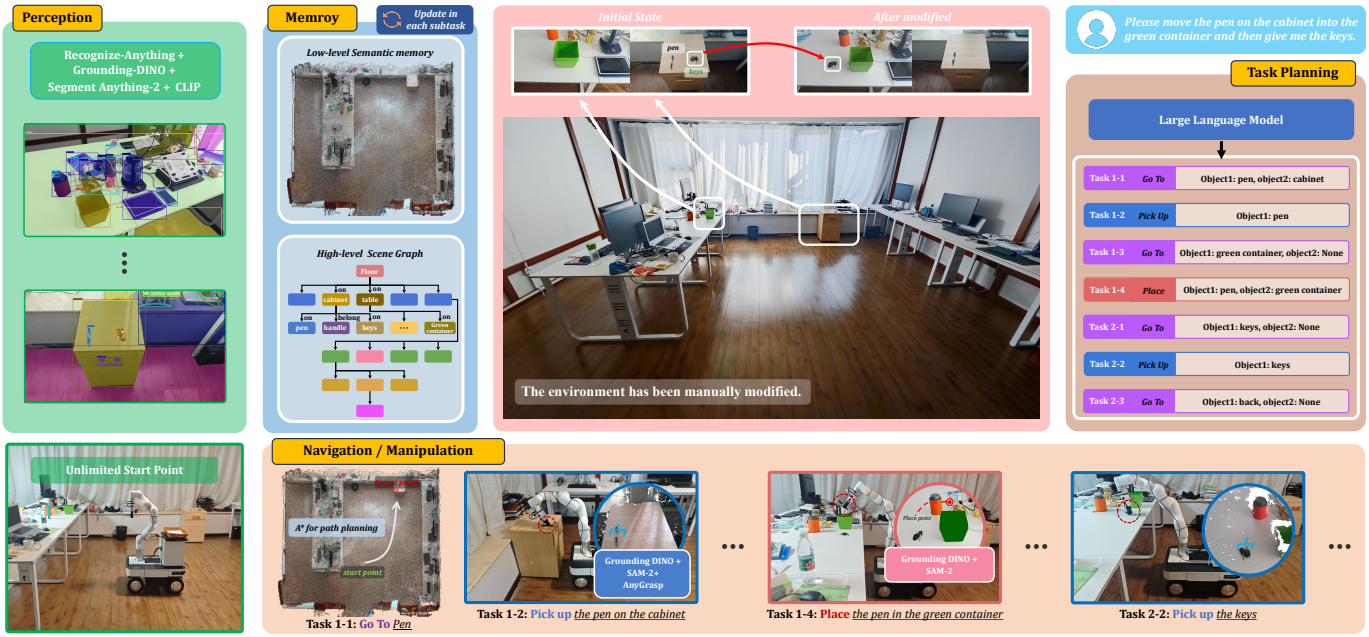


Fig. 1. **Overview of Our DovSG System.** DovSG is a mobile robotic system designed to perform long-term tasks in real-world environments. It can detect changes in the scene during task execution, ensuring that subsequent subtasks are completed correctly. The system consists of five main components: perception, memory, task planning, navigation, and manipulation. The memory module includes a lower-level semantic memory and a higher-level scene graph, both of which are continuously updated as the robot explores the environment. This enables the robot to promptly detect manual changes (e.g., keys being moved from cabinet to table) and make the necessary adjustments for subsequent tasks (such as correctly executing Task 2-2).

***Abstract*—Enabling mobile robots to perform long-term tasks in dynamic real-world environments is a formidable challenge, especially when the environment changes frequently due to human-robot interactions or the robot's own actions. Traditional methods typically assume static scenes, which limits their applicability in the continuously changing real world. To overcome these limitations, we present DovSG, a novel mobile manipulation framework that leverages dynamic open-vocabulary 3D scene graphs and a language-guided task planning module for long-term task execution. DovSG takes RGB-D sequences as input and utilizes vision-language models (VLMs) for object detection to obtain high-level object semantic features. Based on the segmented objects, a structured 3D scene graph is generated for low-level spatial relationships. Furthermore, an efficient mechanism for locally updating the scene graph, allows the robot to adjust parts of the graph dynamically during interactions without the need for full scene reconstruction. This mechanism is particularly valuable in dynamic environments, enabling the robot to continually adapt to scene changes and effectively support the execution of long-term tasks. We validated our system in real-world environments with varying degrees of manual modifications, demonstrating its effectiveness and superior performance in long-term tasks. Our project page is available at: https://bjhyzj.github.io/dovsg-web.**

***Index Terms*—3D scene graph, Long-term Tasks, Mobile Manipulation, Open vocabulary**

[1]School of Mechanical Engineering and Automation, Beihang University.
[2]Centre for Advanced Robotics Technology Innovation, Nanyang Technological University.
[3]School of Information Engineering, Minzu University of China.
[4]Afanti Tech LLC.
✉ **Corresponding to zuoxu_wang@buaa.edu.cn**

I. Introduction

Mobile robot manipulation in real-world environments is increasingly expected to perform complex, long-term tasks that require adaptability and resilience to changes. These dynamic environments are characterized by frequent alterations caused by human activities, robot interactions, or the inherent variability of the surroundings. Traditional robotic systems often fall short in such settings because they typically rely on the assumption of a static or minimally changing environment [1], [2], [3], [4]. This limitation restricts their applicability in real-world scenarios where adaptability is crucial.

In this work, we enhance robotic capabilities by introducing a novel and practical robotic framework, the DovSG system. This framework comprises five key modules: perception, memory, task planning, navigation, and manipulation, as illustrated in Fig. 1. To address the challenge of scene perception, our **perception module** integrates advanced tools such as Recognize-Anything[5], Grounding DINO[6], Segment Anything-2[7], and CLIP[8] to detect objects or components and extract their semantic features. In the **memory module**, each object within

the scene is represented as a node characterized by geometric and semantic features, with the relationships between objects encoded in the graph's edges. The module continuously updates object features and scene graphs by locally refining the areas where the robot interacts, preserving knowledge to support ongoing exploration and exploitation, while avoiding the need to reconstruct the entire scene and enabling more efficient adaptation to dynamic environments.

Our **task planning module** uses the advanced large language models to decompose tasks into manageable subtasks. Then, the **navigation** and **manipulation module** are activated to execute the planned actions. In experiments, we deployed a mobile robot platform in multiple real-world indoor environments, conducting long-term task trials under varying degrees of manual environmental modifications. We demonstrated that DovSG can accurately update the 3D scene graph and performs excellent in long-term tasks and subtasks such as pick-up, place, and navigation. Our **contributions** are as follows:

- We propose a novel robotic framework that integrates dynamic open-vocabulary 3D scene graphs with language-guided task planning, enabling accurate long-term task execution in dynamic and interactive environments.
- We construct dynamic 3D scene graphs that capture rich object semantics and spatial relations, performing localized updates as the robot interacts with its environment, allowing it to adapt efficiently to incremental modifications.
- We develop a task planning method that decomposes complex tasks into manageable subtasks, including pick-up, place, and navigation, enhancing the robot's flexibility and scalability in long-term missions.
- We implement DovSG on real-world mobile robots and demonstrate its capabilities across dynamic environments, showing excellent performance in both long-term tasks and subtasks like navigation and manipulation.

II. Related Works

1) 3D Scene Representations for Perception: 3D scene representation in robotics often follows two main approaches: using foundation models to create 3D structures [9], [10], [11], or combining 2D image and vision-language models [6], [8], [7], [5] to connect with the 3D world, showing impressive results in open-vocabulary tasks [12], [13]. enabling language-guided object grounding [11], [3], [12] and 3D reasoning [14]. However, dense semantic features for each point are redundant, memory-intensive, and not easily decomposable, limiting their use in dynamic robotics applications.

2) 3D scene graphs for memory: 3D scene graphs use a hierarchical graph structure to represent objects as nodes and their relationships as edges within a scene[15]. This approach efficiently stores semantic information in robot memory, even in large and dynamic environments. Recent methods like ConceptGraphs [12] and HOV-SG [16] create more compact and efficient scene graphs by merging features of the same object across multiple views, thereby reducing memory redundancy. However, these methods often assume static environments and overlook scene updates during interactive robot manipulations. RoboEXP [13] addresses this by encoding both spatial relationships and logical associations that reflect the effects of robot actions, enabling the discovery of objects through interaction and supporting dynamic scene updates. Building on this, we have applied these concepts to mobile robots, allowing them to rapidly update memory and adjust the 3D scene graph in real time during dynamic interactions with environments.

3) Large Language Models for Planning: Large language models (LLMs) [17], [18] and VLMs demonstrate significant potential for zero-shot planning in robotics. These models have been used to generate trajectories and plan manipulations, improving robot adaptability. Integrating LLMs with 3D scene graphs offers further opportunities for task planning. In DovSG, we leverage GPT-4 to decompose tasks into subtasks that can be executed through graph memory, enabling robots to handle complex tasks flexibly and adaptively.

4) Indoor Visual Localization for Interaction: Robot interaction relies on accurate tracking of 6-DoF poses using maps constructed from image sequences. Visual relocalization techniques can be categorized into structure-based and learning-based methods. Structure-based approaches employ local descriptors to match 2D pixels to 3D scene coordinates and use PnP algorithms for pose recovery, supported by image retrieval and advanced matching techniques like LightGlue[19] or MatchFormer[20]. While effective for large-scale environments, they struggle in small, static indoor settings due to expanding image and feature databases. Learning-based methods, such as ACE [21] and DSAC* [22], predict poses via direct regression, enabling rapid end-to-end optimization but with limited precision due to reliance on approximate pose estimates. These methods also require scene-specific training, which limits scalability. To achieve fast and accurate relocalization in indoor environments, we combine the strengths of both approaches. We use ACE to estimate an initial pose from 2D images, employ LightGlue to match the most similar historical 2D images and their poses, and then further refine the pose by iterative closest point (ICP[23]).

III. Method

DovSG enables mobile robots to perform long-term tasks in indoor environments by constructing dynamic 3D scene graphs and using large language models for task planning. The process starts with scanning the environment using an RGB-D camera to capture images, followed by open-vocabulary 3D object mapping that detects, associates, and fuses objects into a 3D representation. From these, a 3D scene graph is generated, capturing object relationships and continuously updated when the environment changes. Task planning is performed through language-guided decomposition of long-term tasks into subtasks, which are executed via navigation and manipulation modules.

A. Home Scanning and Coordinate Transformation

1) Home Scanning: Following [4], we captured a video of the room using an Intel Realsense D455 camera, resulting

in a sequence of RGB-D images, $\mathcal{I}_t = I_1, I_2, \ldots, I_t$, $I_t = \langle I_t^{\text{rgb}}, I_t^{\text{depth}} \rangle$ (color and depth), with t as the sequence length. The capture focused on both objects of interest and navigable surfaces, especially around objects and containers.

2) Coordinate Transformation: After data collection, we used DROID-SLAM to estimate the camera poses throughout the sequence, as shown in Fig. 2 (top left). To achieve accurate pose estimation at true scale, we replaced DROID-SLAM's depth prediction with actual depth data from the camera sensor, resulting in camera poses, $\mathcal{P}^{\text{droid}}$. In many SLAM algorithms [24], [25], [26], the first frame's pose is used as the origin, making subsequent poses relative to it, which is often unsuitable for spatial relationships. To normalize the scene, we used the 'floor' as a reference and processed each RGB frame with Grounding DINO and Segment Anything-2 to obtain a floor mask. Since the query only focused on the floor, this process was computationally efficient. Once we obtained the mask, we projected 2D pixels to a 3D point cloud. To align the scene relative to the detected floor, we applied RANSAC to fit a plane to the 3D floor points and computed a transformation matrix T^{floor}, aligning the floor with the global $z = 0$ plane. Each pose in $\mathcal{P}^{\text{droid}}$ was then transformed as: $\mathcal{P} = \mathcal{R}_x T^{\text{floor}} \mathcal{P}^{\text{droid}}$, where $\mathcal{R}_x$ aligns the coordinate system to the x-axis. This yielded the final set of poses, $\mathcal{P} = \{I_1^{\text{pose}}, I_2^{\text{pose}}, \ldots, I_t^{\text{pose}}\}$, with the floor properly aligned. By integrating $\mathcal{P}$ with the image sequence $\mathcal{I}_t$, we obtained $I_t = \langle I_t^{\text{rgb}}, I_t^{\text{depth}}, I_t^{\text{pose}} \rangle$.

B. Open-vocabulary 3D Object Mapping

With the RGB-D sequences and camera poses, we proceed to construct an object-centric 3D representation from RGB-D observations $\mathcal{I}_t = \{I_1, I_2, \ldots, I_t\}$ to objects $\mathbf{O}_t = \{\mathbf{o}_j\}$, $j \in \{1, ..., J\}$ is all object length, each object $\mathbf{o}_j$ is characterized by a 3D point cloud $pcd_{\mathbf{o}_j}$, visual feature $f_{\mathbf{o}_j}^{\text{rgb}}$ and text feature $f_{\mathbf{o}_j}^{\text{text}}$. This map is built incrementally, incorporating each incoming frame I_t into the existing object set $\mathbf{O}_{t-1}$, by either adding to existing objects or instantiating new ones.

1) Open-vocabuary 2D Segmentation: To maximize object recognition in the scene, we first apply the image tagging model Recognize-Anything [5] to each frame I_t, generating a set of object classes $\{\mathbf{c}_{t,i}\}$, $i \in \{1, ..., M\}$ detected in the image. We then use $\{\mathbf{c}_{t,i}\}$, $i \in \{1, ..., M\}$ as input to the 2D detector Grounding DINO to obtain object bounding boxes $\{\mathbf{b}_{t,i}\}$, $i \in \{1, ..., M\}$. Finally, we refine these bounding boxes into object masks $\{\mathbf{m}_{t,i}\}$, $i \in \{1, ..., M\}$ using the advanced segmentation model Segment Anything-2. For each obtained 2D mask $\mathbf{m}_{t,i}$, we extract a cropped image based on its bounding box, as well as an isolated mask image without background, as illustrated in Fig. 2 (bottom-left). We then extract the visual features of each object using two mask-based images with CLIP, and fuse them using a weighted sum method, as described in HOV-SG [16]. This combines the CLIP features from both the cropped target image and the isolated mask image to generate a visual feature descriptor:

$$f_{t,i}^{\text{rgb}} = \text{Embed}(I_t^{\text{rgb}}, \mathbf{b}_{t,i}, \mathbf{m}_{t,i}), \tag{1}$$

while the text descriptor is obtained by:

$$f_{t,i}^{\text{text}} = \text{Embed}(\mathbf{c}_{i,t}), \tag{2}$$

each masked region is then projected into 3D, denoised using DBSCAN clustering with an adaptively computed ε parameter based on the sorted distances to the k-nearest neighbors, and transformed to the map frame, resulting in a point cloud $pcd_{t,i}$, along with unit-normalized semantic feature $f_{t,i}^{\text{rgb}}$ and $f_{t,i}^{\text{text}}$.

2) Object Association: For every newly detected object $\langle pcd_{t,i}, f_{t,i}^{\text{rgb}}, f_{t,i}^{\text{text}} \rangle$, we compute geometric and semantic similarity with respect to all objects $\mathbf{o}_{t\text{-}1,j} = \langle pcd_{\mathbf{o}_j}, f_{\mathbf{o}_j}^{\text{rgb}}, f_{\mathbf{o}_j}^{\text{text}}, \rangle$ in the map that shares any partial geometric overlap. The geometric similarity:

$$\mathbf{s}_{\text{geo}}(i, j) = \text{Nnrate}(pcd_{t,i}, pcd_{\mathbf{o}_j}) \tag{3}$$

is defined as the ratio of the number of points in point cloud $pcd_{t,i}$ that have nearest neighbors in point cloud $pcd_{\mathbf{o}_j}$, within a distance threshold of δ_{nn}, to the total number of points in $pcd_{t,i}$. The visual and text similarity:

$$\mathbf{s}_{\text{vis}}(i, j) = (f_{t,i}^{\text{rgb}})^\top f_{\mathbf{o}_j}^{\text{rgb}}/2 + 1/2 \tag{4}$$

$$\mathbf{s}_{\text{text}}(i, j) = (f_{t,i}^{\text{text}})^\top f_{\mathbf{o}_j}^{\text{text}}/2 + 1/2 \tag{5}$$

is the normalized cosine distance between the corresponding visual descriptors. The overall similarity measure $s(i, j)$ is a weighted sum of the individual similarity measures:

$$s(i, j) = \mathbf{s}_{\text{vis}}(i, j) \cdot \omega_{\text{v}} + \mathbf{s}_{\text{geo}}(i, j) \cdot \omega_{\text{g}} + \mathbf{s}_{\text{text}}(i, j) \cdot \omega_{\text{t}}, \tag{6}$$

where $\omega_{\text{v}} + \omega_{\text{g}} + \omega_{\text{t}} = 1$. We perform object association by a greedy assignment strategy where each new detection is matched with an existing object with the highest similarity score. If no match is found with a similarity higher than δ_{sim}, we initialize a new object.

3) Object Fusion: If a detection $\mathbf{o}_{t\text{-}1,j}$ is associated with a mapped object $\mathbf{o}_j$, we fuse the detection with the map. This is achieved by updating the object's features as

$$f_{\mathbf{o}_j}^{\text{rgb}} = (n_{\mathbf{o}_j} f_{\mathbf{o}_j}^{\text{rgb}} + f_{t,i}^{\text{rgb}})/(n_{\mathbf{o}_j} + 1), \tag{7}$$

where $n_{\mathbf{o}_j}$ is the number of detections that have been associated to $\mathbf{o}_j$ so far; and updating the pointcloud as $pcd_{t,i} \cup pcd_{\mathbf{o}_j}$, followed by down-sampling to remove redundant points.

C. 3D Scene Graph Generation

DovSG constructs a 3D scene graph $\mathcal{G}_t = \langle \mathbf{O}_t, \mathbf{E}_t \rangle$, where and $\mathbf{E}_t = \{\mathbf{e}_k\}$, $k \in \{1, ..., K\}$ represent the sets of objects and edges, respectively. Given the set of 3D objects $\mathbf{O}_T$ obtained from the previous step, we estimate their spatial relationships, i.e., the edges $\mathbf{E}_T$, to complete the 3D scene graph, as shown in Fig. 2 (column 3). Leveraging the transformation of the scene's coordinate system from the previous steps—where the ground plane serves as the origin and the z-axis points upwards—we can efficiently extract the fundamental spatial relationships among the objects. While focusing on these relationships, we first voxelize the point cloud $pcd_{\mathbf{o}_j}$ of each object $\mathbf{o}_j$ in the scene. This reduces computation and storage by converting

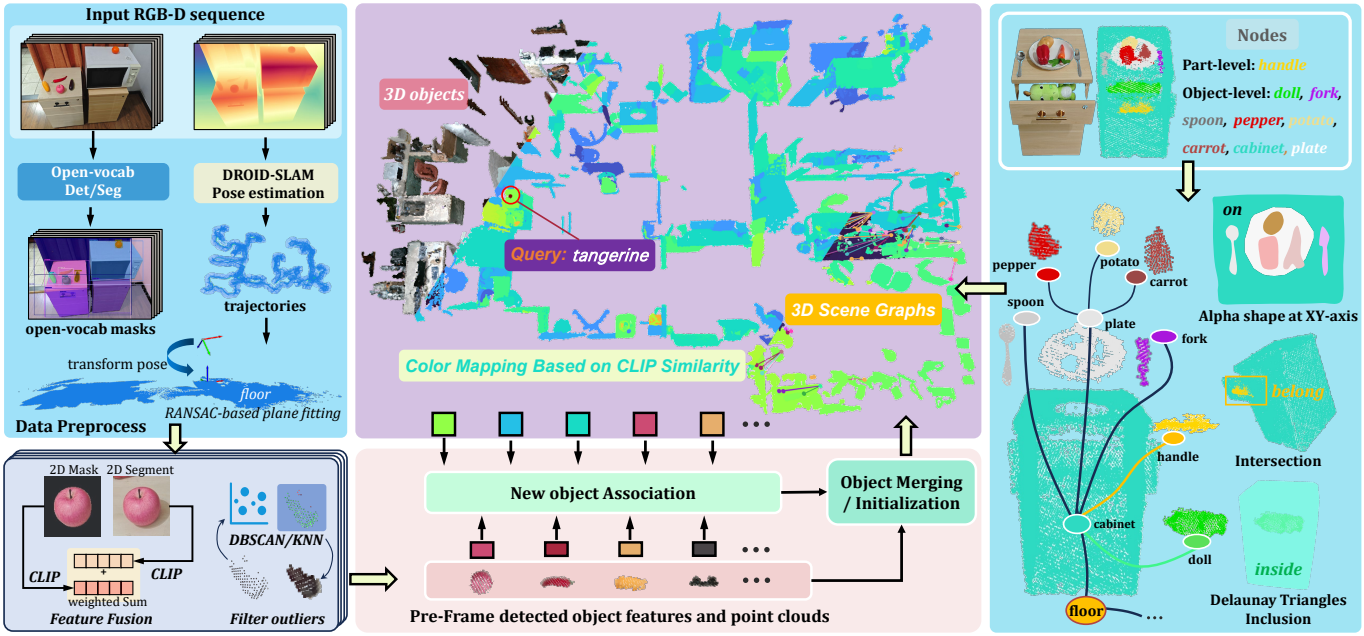


Fig. 2. **Initialization and Construction of 3D Scene Graphs.** We first use the RGB-D-based DROID-SLAM[24] model to predict the pose of each frame in the scene. Then, we apply an advanced Open-Vocal segmentation model to segment regions in the RGB images, extract semantic feature vectors for each region, and project them onto a 3D point cloud. Based on semantic, geometric, and CLIP feature similarities, the same object captured from multiple views is gradually associated and fused, resulting in a series of 3D objects. Next, we infer the relationships between objects based on their spatial positions and generate edges connecting these objects, forming a scene graph. This scene graph provides a structured and comprehensive understanding of the scene, allowing efficient localization of target objects and enabling easy reconstruction and updating in dynamic environments, and supports task planning for large language models.

dense point clouds into sparse voxel grids, ensuring efficient scene updates in future tasks. We focus on three relationships: "on", "belong", and "inside", as mentioned in RoboEXP [13], as these relationships are sufficient for downstream tasks. The "on" relationship denotes a stacking or positional hierarchy between objects, such as an apple being placed on a table. The "belong" relationship captures ownership or attachment between objects, like a refrigerator handle belonging to the refrigerator. Lastly, the "inside" relationship is particularly relevant during the robot's exploration when it opens containers such as drawers or cabinets, revealing that certain objects are contained within others. It is important to note that for "inside" relationship, we limit our focus to small-scale containment between objects, as technically, every object in the scene could be considered "inside" the overall home environment.

D. Dynamic Scene Adaptation

In dynamic indoor interaction scenarios, the layout of the environment changes due to human activities or the robot's task execution. These changes are often invisible to previous approaches[3], [16], [12], and in such environments, if the robot cannot dynamically update its memory, it will soon face failure. To address this issue, we have designed a simple memory update module that can quickly perform local updates to the memory based on new RGB-D observations captured by the robot, as shown in Fig. 3.

1) Relocalization and refinement: Accurate re-localization is crucial for a mobile robot performing manipulations in dynamic environments to maintain an up-to-date scene graph. To ensure precise local updates to the voxelized map, we employ a multi-stage relocalization process that efficiently integrates new observations into the existing scene representation. We begin by training an ACE scene-specific regression MLP using memory $\langle I_i^{\text{rgb}}, I_i^{\text{pose}} \rangle, i \in \{1, ..., t\}$ (see the top left corner in Fig.3). After the robot collects new RGB-D observations I_k for $k \in \{t+1, ..., t+n\}$, where each observation $I_k = \langle I_k^{\text{rgb}}, I_k^{\text{depth}}, I_k^{\text{c2b}} \rangle$ includes the RGB image, depth image, and the camera-to-arm-base transformation, we proceed to estimate the robot's pose within the existing map. Then, leveraging the trained ACE model, we predict a rough global pose I_k^{pose} for each new RGB image I_k^{rgb} by utilizing the prior mapping data $\langle I_i^{\text{rgb}}, I_i^{\text{pose}} \rangle$, where $i \in \{1, ..., t\}$. While ACE provides a coarse alignment, its precision may not suffice for detailed scene updates. To refine these pose estimates, we perform local feature matching using LightGlue [19], identifying the historical image $I_{\hat{k}}^{\text{rgb}}$, $\hat{k} \in \{1, ..., t\}$ with the most feature correspondences to each new observation I_k^{rgb}. This step enhances the robustness of the pose estimation by anchoring it to the most similar view in the prior map. With the matched image pairs, we extract the corresponding point clouds from the RGB-D data and perform multi-scale colored iterative closest point (ICP) alignment. This refinement step minimizes both geometric and photometric discrepancies between the new observations and the existing map, yielding a precise transformation T_k^{icp} that aligns the new observations within the global coordinate frame. The refined poses I_k^{pose} are then updated using the transformation $I_k^{\text{pose}} \leftarrow T^{\text{icp}} I_k^{\text{pose}}$.

2) Remove obsolete indices: This step is to identify and remove obsolete voxels from the memory map. We propose

an efficient method that leverages new RGB-D observations to update the volumetric representation accordingly. Given the set of stored voxel indices, each associated with 3D positions and color information, we process each new observation I_k as follows: (1) We transform the all voxel point cloud (points) $\{pcd_{\mathbf{o}_j}\}$, $j \in \{1, ..., J\}$ into the current camera coordinate by:

$$pcd_{\mathbf{o}_j}^{\text{cam}} = (I_k^{\text{pose}})^{-1} pcd_{\mathbf{o}_j} \tag{8}$$

(2) The transformed points are then projected onto the image plane using the camera's intrinsic parameters to obtain pixel coordinates:

$$\left[\mathbf{u}_j, \mathbf{v}_j\right] = \Pi(pcd_{\mathbf{o}_j}^{\text{cam}}), \tag{9}$$

where Π denotes the projection function. We consider only points that project within the image boundaries and are in front of the camera, that is, $z_j > 0$, z_j is the z-axis coordinates of the $pcd_{\mathbf{o}_j}^{\text{cam}}$. (3) For each valid projected point i, we compute the depth and color difference:

$$\Delta z_i = \left| I_k^{\text{depth}}\left[\mathbf{u}_j^i, \mathbf{v}_j^i\right] - z_j^i \right|, \tag{10}$$

$$\Delta c_i = \left| I_k^{\text{rgb}}\left[\mathbf{u}_j^i, \mathbf{v}_j^i\right] - \mathbf{c}_j^i \right|, \tag{11}$$

where $\mathbf{c}_j$ is the stored color of $pcd_{\mathbf{o}_j}$. (4) We define thresholds δ_z, δ_z', and δ_c for depth and color differences. A point i is marked for deletion if:

$$\text{point } i \text{ is deleted if } \begin{cases} \Delta z_i > \delta_z, \\ \Delta z_i > \delta_z' \text{ and } \Delta c_i > \delta_c. \end{cases} \tag{12}$$

3) Update low-level memory: After the above step, the local scene in the historical low-level memory will be updated to the latest state based on the visual information provided by the new RGB-D observations $I_k, k \in t+1, \ldots, t+n$. We process I_k sequentially in the same way as described in Sec. III-B, and fuse it with the historical $\mathbf{O}_t$ to $\mathbf{O}_{t+1}$. The set $\langle I_k^{\text{rgb}}, I_k^{\text{depth}}, I_k^{\text{pose}} \rangle, k \in t+1, \ldots, t+n$ will also be updated into $\mathcal{I}_{t+1}$, providing new reference viewpoints for the next re-localization to ensure long-term localization accuracy.

4) Update high-Level memory: For updating the scene graph, we adopt a simple local sub-graph-based update strategy to avoid redundant global updates. Specifically, (1) We compare the historical object set $\mathbf{O}_t$ with the updated object set $\mathbf{O}_{t+1}$ to identify the objects that have been deleted or whose points have been changed, forming the set $\mathbf{O}_{\text{affected}}$. (2) Next, we examine the edges in $\mathbf{E}_t$ to find the parent objects of all objects in $\mathbf{O}_{\text{affected}}$ and the child objects of those parent objects. These related objects are also added to $\mathbf{O}_{\text{affected}}$, and all edges associated with $\mathbf{O}_{\text{affected}}$ are removed from $\mathbf{E}_t$. (3) We then identify the objects in $\mathbf{O}_{\text{affected}}$ that still exist in $\mathbf{O}_{t+1}$ and locate any newly added objects in $\mathbf{O}_{t+1}$. Both these types of objects are added to the set $\mathbf{O}_{\text{need_process}}$ for further processing. (4) For each object in $\mathbf{O}_{\text{need_process}}$, we recompute the spatial relationships as described in Sec. III-C. This involves determining the relevant edges $\mathbf{e}$ for each object, updating $\mathbf{E}_{t+1}$ with the new connections and builds $\mathcal{G}_{t+n}$.

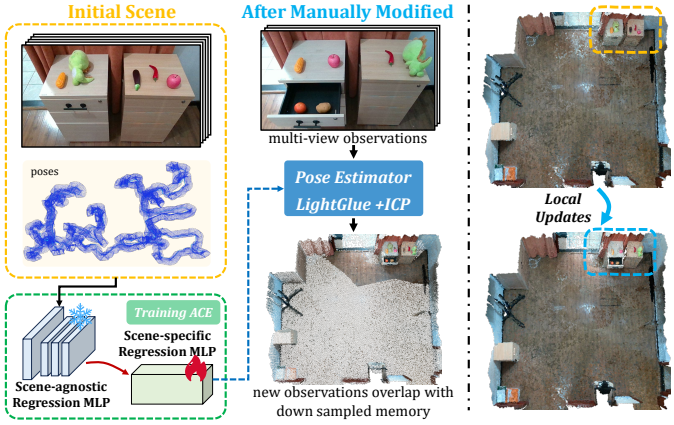


Fig. 3. **Adaptation in interactions with manually modified scenes.** (1) We train the scene-specific regression MLP of the ACE model using RGB images and their poses, making the process highly efficient. (2) After manual scene modification, multi-view observations allow rough global pose estimation via ACE, refined further using LightGlue and ICP. The new viewpoint's point cloud closely aligns with the stored pose. (3) The bottom image shows accurate local updates to the scene based on observations from the new viewpoint.

E. Language-guided Task planning

Benefiting from the system's long-term workability in indoor environments, we integrated the advanced large language model GPT-4o. Based on class-agnostic and highly extensible prompt text descriptions, we decompose long-term tasks described in natural language into multiple subtasks that the robot can easily adapt to. Each subtask output by GPT consists of an "action_name" and multiple "object_name", which are directly extracted from the description and maintain the same level of abstraction as the described objects, ensuring that the original meaning is not distorted during task decomposition.

F. Navigation

1) Localization: Before initiating each navigation task, we first determine the robot's precise pose as described in Sec. III-D1, establishing its position in the world coordinate system (start point). To locate the target object(s), we utilize CLIP to obtain embeddings of the specified object names. If only object A is specified, we compute its CLIP embedding and compare it with the embeddings of all objects in the scene using cosine similarity. The object with the highest similarity score is identified as the target location of A (as illustrated in the upper section of Fig. 2). For tasks involving a spatial relationship between two objects (e.g., "object A is on object B"), we compute CLIP embeddings for both A and B. We then compare these embeddings with those of the scene objects to obtain similarity scores. For each object, we select the top-k most similar scene objects (top-k A and top-k B, respectively). Next, we calculate the Euclidean distances between each pair of candidate locations from top-k A and top-k B. The pair with the shortest distance is deemed the most probable configuration, and the location of the corresponding A candidate is selected as the target point.

2) Mobile control: Once the target location is determined, we use the A*[27] algorithm to generate a collision-free navigation path from the start point to the target point. The

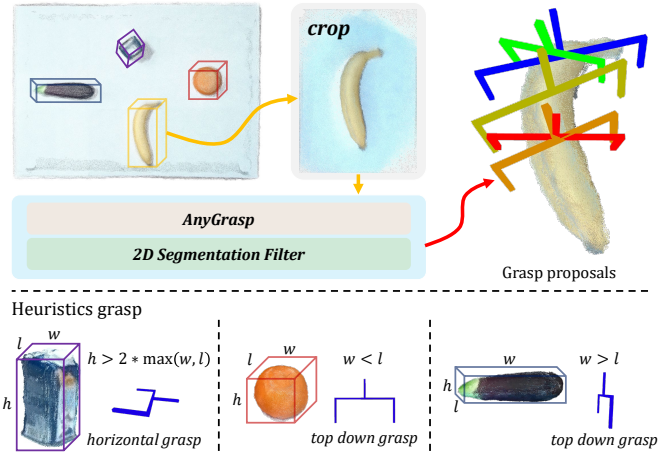


Fig. 4. **Two proposed grasp strategies in DovSG**. In the first row, we cropped the point cloud input into anyGrasp within a certain range around the target object, allowing anyGrasp to focus more on the target object without compromising the generation of collision-free grasps. Furthermore, we filtered the grasps based on translational and rotational costs, with the red grasps indicating the highest confidence. In the second row, we show our heuristic grasp strategy, which leverages the object's bounding box information to rotate and select the most appropriate grasp orientation.

robot then follows this path using a PID controller to ensure accurate and smooth navigation.

G. Manipulation

Once the robot arrives at the target location, we employ the method described in Ok-Robot [3] to perform the pick-and-place manipulation. At first, the robot's camera is oriented toward the 3D location of the object, as provided by the semantic memory, to capture RGB-D images.

1) Pick up: To focus the AnyGrasp model on the target object, we first preprocess the point cloud by cropping it to a region around the target object, based on its detected bounding box. This step refines the input for AnyGrasp, leading to more accurate and efficient grasp predictions. After generating candidate grasps, we apply cost-based filtering to select the best option. The robot executes the grasp with the highest confidence, leveraging additional segmentation from Grounding DINO and Segment Anything-2 for precise targeting. Furthermore, we introduce a heuristic grasp strategy that uses the object's bounding box information to rotate and align the gripper for the most suitable grasp orientation, as shown in Fig. 4. This heuristic strategy is only activated when AnyGrasp can't provide a suitable grasp, ensuring optimal interaction with the object's geometry

2) Place: We first obtain the point cloud pcd_a of the target object using Grounding DINO and Segment Anything-2 (SAM2), as described in Sec. III-B1. This point cloud is then transformed into the robot's base coordinate frame. Next, we compute the median coordinates x_m and y_m, and determine the drop height as follows:

$$z_{\max} = 0.1 + \max\left(z \mid 0 \leq x \leq x_m, |y - y_m| < 0.1\right), \quad (13)$$

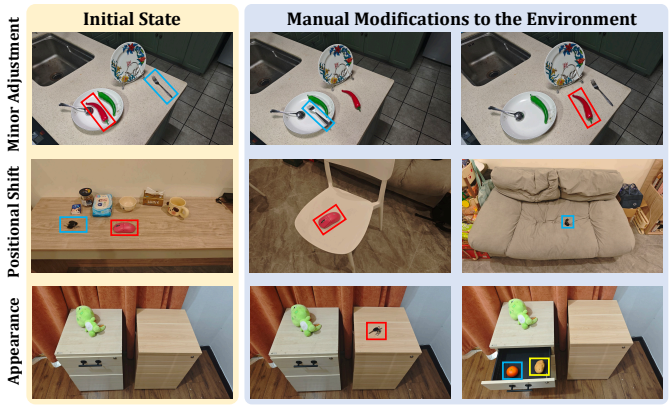


Fig. 5. **Degrees of environmental modifications.** The left column shows the initial state of the scene, while the two columns on the right represent the state of the scene after manual modifications.

where $(x, y, z) \in pcd_a$. A buffer of 0.1 is added to account for potential collisions. The robot executes the placement manipulation at the computed coordinates $(x_m, y_m, z_{\max})$.

IV. Experiments

In this section, We evaluate DovSG's performance in dynamic, real-world environments to answer two key questions: (1) How well does our system adapt to changes by updating the dynamic scene graph? (2) How effectively does this facilitate the completion of consecutive tasks without manual resets?

A. Robot Setups

We used a real-world setup with a UFACTORY xARM6 robotic arm on an Agilex Ranger Mini 3 mobile base, equipped with a RealSense D455 camera for perception and a basket for item transport.

B. Task and Environment Setups

To verify whether our method enables a robot to complete long-term tasks in dynamic environments by continuously updating the scene graph, we designed an innovative experiment based on two real-world rooms. The experiment involved human interactions that modified object positions, added new objects, or revealed hidden ones, simulating a dynamic environment. Each experiment begins with a 3D scene graph as the foundation and includes two consecutive tasks. Before the robot executes the first task, we manually adjusted the scene by changing object positions, adding objects, or revealing hidden ones related to the second task, ensuring these changes could be detected during the execution of the first task.

After completing a task, instead of being repositioned to an initial start point, the robot continues to the next task from its current state, simulating real-world scenarios with continuous environmental changes caused by human interference. This approach ensures the robot not only completes individual tasks but also updates the scene graph in response to changes, maintaining an accurate understanding of the environment and efficiently completing subsequent tasks.

To evaluate the robot's ability to detect and adapt to environmental changes, we categorized the modifications to

the objects in the second task into three levels, ranging from subtle to significant changes(as shown in Fig. 5): (1) ***Minor Adjustment:*** A slight modification of the object's position that remains detectable from the object's original location. For instance, if the robot searches for an object in its original position, it may still be visible within the robot's field of view, even if it's been slightly moved. (2) ***Appearance:*** An object that was previously hidden or did not exist in the scene becomes visible due to human interaction. This could involve opening a closed drawer to reveal items inside, or a human placing a key on the table that was not there before. This change introduces new nodes to the 3D scene graph. (3) ***Positional Shift:*** A modification where the object's location changes significantly enough that it can no longer be detected from its original position. This could involve relocating the object from one surface to another, altering its relationship with surrounding objects in the scene graph. Each task is evaluated across 40 trials, with 20 trials per room, where objects or their positions are randomized in every trial.

TABLE I
QUANTITATIVE RESULTS ON DYNAMIC SCENE ADAPTATION AND SCENE GRAPH GENERATION.

Task	Minor Adjustment		Appearance		Positional Shift	
Metrics	GPT-4o	Ours	GPT-4o	Ours	GPT-4o	Ours
SCDA	40.6%	**95.3%**	63.9%	**92.0%**	65.9%	**94.3%**
SGA	54.6%	**89.5%**	51.1%	**85.2%**	46.5%	**82.9%**

C. Baselines

For long-term task execution, we implemented Ok-Robot [3] on our mobile robot as a comparison method. In addition, to demonstrate our approach's adaptability to changing environments and the effectiveness of scene graph prediction, we also evaluated scene changes and scene graph predictions. Using GPT-4o as the base model, enhanced with chain-of-thought (CoT) reasoning [28], similar to the method proposed by Jiang et al. [13], the model takes RGB observations from the robot, matches them to the most similar image in memory, detects differences, and predicts the scene graph based on the new RGB observation.

TABLE II
SUCCESS RATE OF LONG-TERM TASKS AND SUBTASKS

Task	Method	Minor Adjustment	Appearance	Positional Shift	**Total(%)**
Pick up	Ok-Robot	39 / 51	35 / 52	36 / 49	72.3%
	Ours	53 / 68	54 / 68	53 / 66	**79.2%**
Place	Ok-Robot	24 / 30	22 / 28	26 / 32	80.0%
	Ours	33 / 41	34 / 42	33 / 39	**81.9%**
Navigation	Ok-Robot	89 / 103	82 / 100	81 / 101	82.8%
	Ours	121 / 122	118 / 124	111 / 122	**95.1%**
Long-term	Ok-Robot	6 / 40	0 / 40	0 / 40	5.0%
	Ours	16 / 40	14 / 40	10 / 40	**33.3%**

D. Evaluation

To thoroughly evaluate DovSG's adaptability to dynamic environments and its effectiveness in long-term tasks, we involved human evaluators to modify the environment, construct ground truth scene graphs, and determine task execution success. Our evaluation focuses on three key aspects: ***Dynamic Scene Adaptation and Scene Graph Generation:*** We recorded RGB observation data during the robot's task execution and compared DovSG's performance with baseline methods in detecting scene changes and generating scene graphs. ***Long-Term Tasks Evaluation:*** In addition to assessing the overall success rate of long-term tasks, we evaluated the success rates of individual subtasks such as picking, placing, and navigation. This detailed evaluation provides a comprehensive analysis of the effectiveness of our method across different components of task execution. The three main metrics used for evaluation are as follows: **(1) Scene Change Detection Accuracy (SCDA):** This metric measures the accuracy of detecting changes in the observed scene and matching them to the corresponding memory-stored RGB information. **(2) Scene Graph Accuracy (SGA):** This metric evaluates whether the generated scene graph matches the ground truth (GT) scene graph. If the final scene graph perfectly matches the GT version, the experiment is considered successful and assigned a score of 1, otherwise 0. **(3) Task Success Rate (TSR):** This metric represents the overall task completion success rate. For long-term tasks, success is only counted if all subtasks are completed successfully.

E. Results

As shown in Tab. I, in the SCDA evaluation, GPT-4o often struggles with inconsistencies between the scope of historical observations and new observations, which introduces a significant amount of noise in the responses, resulting in subpar performance. This inconsistency also makes accurate detection under Minor Adjustment scenarios very challenging. In contrast, DovSG, supported by precise re-localization, can accurately identify the voxel index where changes have occurred in the scene, significantly outperforming the baseline. In the SGA evaluation, both approaches generate scene graphs based on new observations and identify only three types of relationships: "on", "belong", and "inside". As a result, the performance in scenarios involving these three types of modifications is quite similar. The main issue with our method arises when the same object is broken into multiple nodes, which affects the accuracy of the scene graph. On the other hand, GPT-4o struggles with recognizing relationships accurately or redundantly defining objects, leading to erroneous results.

In Tab. II, we have the fellow observations: (1) In the Minor Adjustment environment, despite the object being slightly moved, it remains within the robot's field of view, which allows a high likelihood of navigating near the target. Therefore, the success rate is significantly higher compared to Positional Shift and Appearance. In the case of Positional Shift, the residual effect of CLIP features can sometimes cause the robot to mistakenly perceive the object's location, leading it to navigate to the historical position and ultimately resulting in navigation failure. On the other hand, for Appearance, where new objects appear, the robot does not face the challenge of misjudging the original object's location, which generally leads to a higher success rate compared to Positional Shift. (2) Compared to Ok-Robot, DovSG significantly improved the "Pick up" success

TABLE III
EFFIECIENCY COMPARISON BETWEEN OK-ROBOT AND DovSG

Method	Resolution (cm)	Memory (GB)	Time (min)
Ok-Robot	1	2	20
Ours	1	**0.15**	**1**

rate by employing two proposed grasp strategies, focusing specifically on the target object. Additionally, we optimized the selection of grasp candidates to reduce environmental interference, ensuring that the robot chooses the optimal grasp. As a result, our method notably outperforms Ok-Robot in terms of pick-up success. (3) For long-term tasks, integrating the dynamic scene graph enables our method to significantly outperform the static-scene assumption used by Ok-Robot. Although Ok-Robot might occasionally succeed in locating the correct object in environments with minor changes, such as Minor Adjustment, it struggles with more substantial changes, such as Positional Shift or Appearance, due to its inability to perform partial updates to the scene, rendering success nearly impossible in these scenarios.

Tab. III further demonstrates the memory consumption and time required for both recording a scene and performing an update at a resolution of 1 cm for DovSG and Ok-Robot. Notably, because Ok-Robot cannot perform partial updates, the table only presents the time needed for a full scene update. The results show that Ok-Robot's memory and time requirements are at least 20 times greater than those of DovSG.

V. CONCLUSION

In this paper, we introduce DovSG, an innovative framework designed to enable mobile robots to perform long-term tasks in dynamic environments by continuously updating 3D scene graphs. Our approach enables the robot to adapt to changes caused by human interaction or by the robot's own task execution. Through extensive experiments in indoor environments with varying degrees of modifications, we demonstrate that DovSG can effectively detect changes in the visible scene and accurately update the local 3D scene graph. This ensures the precise execution of long-term tasks without being affected by cumulative temporal errors. By evaluating long-term tasks and subtasks such as pickup, place, and navigation, our results highlight the robustness and effectiveness of DovSG in handling the complexity of dynamic real-world scenes.

REFERENCES

[1] A. Brohan, N. Brown, and e. a. Justice Carbajal, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," 2023. [Online]. Available: https://arxiv.org/abs/2307.15818

[2] D. Driess, F. Xia, and e. a. Sajjadi, Mehdi S. M., "Palm-e: An embodied multimodal language model," in *arXiv preprint arXiv:2303.03378*, 2023.

[3] P. Liu, Y. Orru, J. Vakil, C. Paxton, N. M. M. Shafiullah, and L. Pinto, "Ok-robot: What really matters in integrating open-knowledge models for robotics," 2024. [Online]. Available: https://arxiv.org/abs/2401.12202

[4] T. Gilles, S. Sabatini, D. Tsishkou, B. Stanciulescu, and F. Moutarde, "Gohome: Graph-oriented heatmap output for future motion estimation," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 9107–9114.

[5] Y. Zhang, X. Huang, J. Ma, Z. Li, Z. Luo, Y. Xie, Y. Qin, T. Luo, Y. Li, S. Liu, *et al.*, "Recognize anything: A strong image tagging model," *arXiv preprint arXiv:2306.03514*, 2023.

[6] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, "Grounding dino: Marrying dino with grounded pre-training for open-set object detection," 2024. [Online]. Available: https://arxiv.org/abs/2303.05499

[7] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, "Sam 2: Segment anything in images and videos," 2024. [Online]. Available: https://arxiv.org/abs/2408.00714

[8] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," 2021. [Online]. Available: https://arxiv.org/abs/2103.00020

[9] W. Shen, G. Yang, A. Yu, J. Wong, L. P. Kaelbling, and P. Isola, "Distilled feature fields enable few-shot language-guided manipulation," 2023. [Online]. Available: https://arxiv.org/abs/2308.07931

[10] J. Kerr, C. M. Kim, K. Goldberg, A. Kanazawa, and M. Tancik, "Lerf: Language embedded radiance fields," 2023. [Online]. Available: https://arxiv.org/abs/2303.09553

[11] Y. Zheng, X. Chen, Y. Zheng, S. Gu, R. Yang, B. Jin, P. Li, C. Zhong, Z. Wang, L. Liu, C. Yang, D. Wang, Z. Chen, X. Long, and M. Wang, "Gaussiangrasper: 3d language gaussian splatting for open-vocabulary robotic grasping," 2024. [Online]. Available: https://arxiv.org/abs/2403.09637

[12] Q. Gu, A. Kuwajerwala, and e. a. Sacha Morin, "Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning," 2023. [Online]. Available: https://arxiv.org/abs/2309.16650

[13] H. Jiang, B. Huang, R. Wu, Z. Li, S. Garg, H. Nayyeri, S. Wang, and Y. Li, "Roboexp: Action-conditioned scene graph via interactive exploration for robotic manipulation," 2024. [Online]. Available: https://arxiv.org/abs/2402.15487

[14] S. Peng, K. Genova, C. M. Jiang, A. Tagliasacchi, M. Pollefeys, and T. Funkhouser, "Openscene: 3d scene understanding with open vocabularies," 2023. [Online]. Available: https://arxiv.org/abs/2211.15654

[15] R. Wu, K. Xu, C. Liu, N. Zhuang, and Y. Mu, "Localize, assemble, and predicate: Contextual object proposal embedding for visual relation detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 07, 2020, pp. 12 297–12 304.

[16] A. Werby, C. Huang, M. Büchner, A. Valada, and W. Burgard, "Hierarchical open-vocabulary 3d scene graphs for language-grounded robot navigation," *Robotics: Science and Systems*, 2024.

[17] OpenAI, "Gpt-4 technical report," 2023.

[18] Google, "Gemini: A family of highly capable multimodal models," 2023.

[19] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local Feature Matching at Light Speed," in *ICCV*, 2023.

[20] Q. Wang, J. Zhang, K. Yang, K. Peng, and R. Stiefelhagen, "Matchformer: Interleaving attention in transformers for feature matching," 2022. [Online]. Available: https://arxiv.org/abs/2203.09645

[21] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated coordinate encoding: Learning to relocalize in minutes using rgb and poses," 2023. [Online]. Available: https://arxiv.org/abs/2305.14059

[22] E. Brachmann and C. Rother, "Visual camera re-localization from rgb and rgb-d images using dsac," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2022.

[23] P. J. Besl and N. D. McKay, "Method for registration of 3-d shapes," in *Sensor fusion IV: control paradigms and data structures*, vol. 1611. Spie, 1992, pp. 586–606.

[24] Z. Teed and J. Deng, "DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras," *Advances in neural information processing systems*, 2021.

[25] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: an open-source SLAM system for monocular, stereo and RGB-D cameras," *IEEE Transactions on Robotics*, vol. 33, no. 5, pp. 1255–1262, 2017.

[26] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, "Nice-slam: Neural implicit scalable encoding for slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[27] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[28] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," 2023. [Online]. Available: https://arxiv.org/abs/2201.11903